
GenAI Enterprise Knowledge Assistant — Engineering Specification

This is the single source of truth for this project: what it is, how it's built, who owns what, and the exact contracts every piece must follow. It is written to be handed to an AI coding assistant along with a statement of which part a developer owns, so the assistant can generate correct, consistent code without inventing anything that would clash with the other seven people's work.

0. How to use this document (read this section first)

You are being given this document by one developer on an 8-person team, each building a different, non-overlapping part of the same system. Before writing any code:

1. Ask the developer which tag they are (**P1** through **P8**) if they haven't said so. Look up that tag in Section 11 to find exactly which files they own and what those files need to do.
 2. Only write code for files owned by that tag, per the ownership table in Section 3. Never create, rename, restructure, or modify a file owned by a different tag — if their work needs something from another file, call the function or hit the endpoint using the exact signature given in Section 6, don't open that file.
 3. Treat the database schema (Section 4), API contracts (Section 5), and function signatures (Section 6) as fixed. Do not change a field name, add an undeclared field, or alter a response shape — every other developer's code is being generated against these exact shapes, independently, in a separate session. Any deviation breaks integration.
 4. Build **Priority 1** work before **Priority 2** work for that tag, unless the developer explicitly says Priority 1 is already done and merged. If asked for Priority 2 code, still check whether it depends on another tag's Priority 1 or Priority 2 output (Section 6 notes this) and flag that dependency rather than silently stubbing it.
 5. Follow the coding conventions in Section 10 exactly — this is what keeps eight independently-generated codebases mergeable into one.
 6. If something a developer asks for isn't covered by this document (a new field, a new endpoint, a new file), say so and ask them to confirm the addition rather than inventing it — an invented contract is exactly what causes merge conflicts and integration breaks between people who can't see each other's sessions.
 7. When you finish a piece of work, check it against Section 12 before handing it back.
-

1. Project Overview

What it is: an internal Q&A assistant over institutional policies, syllabi, circulars, and process documents — hundreds of PDFs spread across shared folders. Keyword search today returns whole documents instead of answers; staff and students repeatedly ask the same questions and sometimes act on outdated versions of a policy.

What the system must do: - Find the right passage, not just the right document. - Answer only from what it actually retrieved, with citations. - Know when it doesn't know, and refuse rather than guess. - Detect when two "current" versions of a document disagree, and surface both instead of silently picking one. - Do all of this per tenant (enterprise), behind a real frontend, with usage and feedback captured.

Why this architecture, in one paragraph: naive chunk-and-embed RAG underperforms on this corpus for two structural reasons. First, policy documents are heavily structured (numbered sections, nested clauses, cross-references) and a flat vector index throws that structure away — heading-aware chunking plus a coarse routing step (find the right document and section before searching within it) restores it. Second, these documents are full of exact identifiers (GR numbers, form codes, section numbers) that dense embeddings are structurally bad at matching — hybrid retrieval (dense + BM25, fused with Reciprocal Rank Fusion) covers that gap. On top of retrieval accuracy, metadata filtering (department, doc type, effective date, version status) and refusal/conflict detection are first-class requirements, not nice-to-haves — a confidently wrong answer on a policy question is worse than no answer.

Non-functional requirements — every file must respect these, not just one owner's: - **Traceability:** every answer must trace back to a specific (document, section, chunk). Metadata tagging at ingestion, retrieval responses, and generation citations must all carry these ids through unbroken. - **Tenant isolation:** every table and every query is scoped by `tenant_id`, no exceptions. No retrieval or generation step may ever see data across tenants. - **Bounded cost as the corpus grows:** the expensive operations — LLM-based routing and cross-encoder reranking — must only ever run on an already-narrowed candidate set, never the full corpus. - **Graceful degradation:** every Priority 2 stage must have a working Priority 1 fallback, so the system stays fully functional (just less precise) if a given Priority 2 piece isn't built or fails. Priority 2 code should be written so a missing dependency or an exception in it falls back to the Priority 1 behavior rather than crashing the request.

2. The Pipeline — stages, priority, and ownership

Stage 0 – Ingestion (offline, runs once per document, before any query)
 Stage 1 – Query Understanding & Rewriting
 Stage 2 – Coarse Routing (which document, which section)
 Stage 3 – Fine-Grained Hybrid Retrieval (scoped to Stage 2's output)
 Stage 4 – Reranking
 Stage 5 – Grounded Generation, Citation, Refusal / Conflict Check

Priority definitions used throughout this document: - **Priority 1** — must work end to end; this is the demo's spine. Build this first, always. - **Priority 2** — differentiators; adds real accuracy/UX value, built once Priority 1 is solid. Every Priority 2 item has a defined Priority 1 fallback so the system works without it. - **Priority 3** — out of scope for this build (semantic caching, analytics dashboard, incremental re-indexing automation). Mentioned only so no one accidentally builds it.

Stage	What	Priority	Owner	File(s)
0	OCR-parse → structured markdown → heading-aware chunk → embed → store	Priority 1	P1	ingestion/ocr.py , chunker.py , metadata_tagger.py , loader.py
0	Per-document summary (short LLM summary, generated once per doc)	Priority 2	P1	ingestion/summarizer.py
0	Section tree extraction (heading hierarchy)	Priority 2	P1	ingestion/section_tree.py
0	Acronym/entity glossary, auto-built from the corpus	Priority 2	P1	ingestion/glossary_builder.py
1	Query rewriting: acronym expansion, metadata predicate extraction, compound-question decomposition, separate BM25/dense phrasings, clarify-then-fallback	Priority 2 (Priority 1 fallback: use the raw query as-is for both BM25 and dense, no auto filter)	P4	generation/query_rewriter.py
2a	Document-level candidate selection (metadata filter + summary match → top 3-5 docs)	Priority 2	P2	retrieval/routing.py
2b	Section-level tree reasoning (LLM picks which section(s) govern the question)	Priority 2	P2	retrieval/routing.py
3	Dense retrieval (pgvector) + metadata filter as a hard constraint	Priority 1	P2	retrieval/dense_retrieval.py
3	BM25 (tsvector) + dense, fused with Reciprocal Rank Fusion	Priority 2	P2	retrieval/hybrid_retrieval.py
4	Cross-encoder reranking of the fused top-k down to top-n	Priority 2 (no Priority 1 version — skipped entirely until built)	P3	retrieval/reranker.py
5	Grounded generation with inline citation; refusal on low confidence	Priority 1	P4 + P5	generation/generator.py , grounding.py
5	Version-conflict detection — two “current” docs disagree, surface both	Priority 2	P5	generation/conflict_detector.py

3. Repo Structure — one owner per file

```

repo/
├── backend/app/
│   ├── main.py                [P2] – FastAPI app, wires all routers together
│   ├── config.py              [P2] – env vars, settings (Settings object; no raw os.environ elsewhere)
│   ├── database.py             [P2] – SQLAlchemy async engine/session
│   ├── schemas.py              [P2] – Pydantic models for every shape in Sections 4-6
│   └── deps.py                 [P2] – shared deps (get_db, get_current_user)
│
│   ├── ingestion/              [P1 – whole folder]
│   │   ├── ocr.py               – Stage 0: document parsing (model in Section 7)
│   │   ├── chunker.py           – Stage 0: heading-aware chunking
│   │   ├── metadata_tagger.py   – Stage 0: department/doc_type/date/version
│   │   ├── summarizer.py        – Stage 0, Priority 2: per-doc summary
│   │   ├── section_tree.py      – Stage 0, Priority 2: heading hierarchy
│   │   ├── glossary_builder.py  – Stage 0, Priority 2: acronym glossary
│   │   ├── loader.py            – writes everything into the DB
│   │   └── run_ingestion.py     – runs the full pipeline
│   │
│   ├── retrieval/
│   │   ├── embeddings.py        [P3] – Stage 0/3: embeddings API wrapper
│   │   ├── indexer.py           [P3] – Stage 0: batch-embeds chunks, stores vectors
│   │   ├── reranker.py          [P3] – Stage 4, Priority 2: cross-encoder rerank
│   │   ├── dense_retrieval.py   [P2] – Stage 3: pgvector search + metadata filter
│   │   ├── hybrid_retrieval.py  [P2] – Stage 3, Priority 2: BM25 + RRF fusion
│   │   ├── routing.py           [P2] – Stage 2, Priority 2: coarse doc/section routing
│   │   └── router.py            [P2] – exposes POST /retrieve
│   │
│   ├── generation/
│   │   ├── prompts.py           [P4] – prompt templates
│   │   └── generator.py         [P4] – Stage 5: calls LLM, builds cited answer, orchestrates the full pipeline call
│   │
│   └── order (Section 6)
│       ├── query_rewriter.py    [P4] – Stage 1, Priority 2
│       ├── router.py            [P4] – exposes POST /chat (streams via SSE)
│       ├── grounding.py         [P5] – Stage 5: confidence scoring + refusal
│       └── conflict_detector.py  [P5] – Stage 5, Priority 2: version-conflict
│
│   ├── auth/                    [P6 – whole folder]
│   │   ├── models.py, security.py, tenancy.py, router.py
│   │
│   └── eval/                     [P8 – whole folder]
│       ├── gold_set.py, harness.py, report.py, run_eval.py
│
├── frontend/src/
│   ├── main.tsx, App.tsx, api/client.ts [P7]
│   ├── auth/                     [P6 – whole folder]
│   └── chat/                       [P7 – whole folder]
│
└── eval/gold_qa.json              [P8]

```

The rule: need something from a file you don't own? Call the function or hit the endpoint using the exact signature in Section 6. Never edit someone else's file.

4. Database Schema — fixed

enterprises	id, name, created_at
users	id, tenant_id, email, password_hash, role
documents	id, tenant_id, title, department, doc_type, effective_date, version_status, source_path, summary, -- Priority 2 (Stage 0), used by Stage 2a section_tree -- Priority 2 (Stage 0, jsonb), used by Stage 2b
chunks	id, document_id, tenant_id, text, section_path, embedding (vector), text_search (tsvector), department, doc_type, effective_date, version_status
glossary	id, tenant_id, term, expansion -- Priority 2 (Stage 0), used by Stage 1
queries	id, tenant_id, user_id, raw_query, rewritten_query, routed_doc_ids, retrieved_chunk_ids, confidence_score, answered_or_refused, created_at
feedback	id, query_id, thumbs_up_down, comment

Owner: P2. Nobody else writes migrations.

5. API Contracts — fixed

POST /auth/login [P6] → { email, password, tenant_code } → { access_token, tenant_id, user_id, role }

GET /auth/me [P6] → { user_id, tenant_id, email, role }

POST /retrieve [P2]

```
request: { query, tenant_id, top_k, filters: { department, doc_type, version_status },
          scoped_sections: [ { document_id, section_path } ] } -- optional, from Stage 2; omit/null in the Priority 1 path
response: { chunks: [ { chunk_id, document_id, text, section_path, score,
                       department, doc_type, effective_date, version_status, source_path } ] }
```

POST /chat [P4] — streams over SSE, three possible event types

```
request: { query, tenant_id, conversation_id }
token event: { type: "token", content }
clarify event: { type: "clarify", question } -- Priority 2 only: asked when no role/department signal is found
final event: { type: "final", answer, citations: [ { chunk_id, document_id, section_path, source_path } ],
              confidence, refused, refusal_reason, conflict }
```

POST /feedback [P6 — Priority 2] → { query_id, thumbs_up_down, comment } → { status: "ok" }

All error responses use one shared envelope, regardless of endpoint: { error: string, detail: string }.

6. Function-Level Contracts and Call Order

- `embeddings.py` [P3]: `embed_text(text: str) -> list[float]`, `embed_batch(texts: list[str]) -> list[list[float]]`.
- `dense_retrieval.py` [P2]: `retrieve_chunks(query_embedding, tenant_id, filters, top_k, scoped_sections=None) -> list[ChunkResult]` — if `scoped_sections` is given, search is restricted to those (`document_id`, `section_path`) pairs; otherwise it searches the whole metadata-filtered corpus (the Priority 1 path).
- `routing.py` [P2]: `route_query(rewritten_query, tenant_id) -> list[{document_id, section_path}]` — 2a (filter + summary match → 3-5 candidate docs) then 2b (LLM reads each candidate's `section_tree`, picks the governing section). Returns 1-3 pairs, typically. Depends on P1's `summary / section_tree` columns being populated.
- `query_rewriter.py` [P4]: `rewrite(raw_query, tenant_id) -> { expanded_query, metadata_filters: {department, doc_type, role, version_status}, bm25_variant, dense_variant, sub_queries, needs_clarification, clarifying_question }`. Depends on P1's glossary table for acronym expansion.
- `reranker.py` [P3]: `rerank(query, chunks, top_n=5) -> list[ChunkResult]` — cross-encoder scores (query, chunk_text) jointly on the fused top-~25, returns the reordered top-~5.
- `grounding.py` [P5]: `decide_refusal(query, top_chunks, draft_answer) -> { refused, reason, confidence, conflict }`.
- `conflict_detector.py` [P5]: `check_conflict(top_chunks) -> { conflict, conflicting_chunks }` — called from inside `decide_refusal`.
- `security.py` [P6]: `get_current_user(token) -> CurrentUser` — imported once into P2's `deps.py`.

Call order inside `generation/generator.py` [P4] — the actual pipeline. Build the Priority 1 version first; add each Priority 2 step behind a check that falls back cleanly if the step isn't built or raises:

1. If `query_rewriter` is available: call `rewrite()`. If `needs_clarification` is true, stream the `clarify` event and stop. Otherwise use its output. If not available: `expanded_query = dense_variant = bm25_variant = raw query`, `metadata_filters = whatever the UI set manually`, `sub_queries = []`.
2. For each sub-query (normally one): if `routing.py` is available, call `route_query()` for `scoped_sections`; if not, `scoped_sections = None`.
3. Call `dense_retrieval.retrieve_chunks(embed(dense_variant), tenant_id, metadata_filters, top_k=25, scoped_sections)`. If `hybrid_retrieval.py` is available, also run BM25 on `bm25_variant` and fuse with RRF.
4. If `reranker.py` is available, call `rerank(expanded_query, chunks, top_n=5)`; if not, take the first 5 chunks as-is.
5. Merge results across sub-queries if there was more than one.
6. Draft the answer: LLM answers using **only** the top chunks' content, with inline citations like `[Leave Policy 2025, Section 3.2.2]`.
7. Call `grounding.decide_refusal(query, top_chunks, draft_answer)` (which internally calls `conflict_detector.check_conflict` if that file exists). Build the final response per Section 5.

7. Models

Component	Model	Called from	Notes
LLM (generation, rewriting, routing/reasoning, self-confidence)	OpenAI API — gpt-4o-mini	<code>generator.py</code> , <code>query_rewriter.py</code> , <code>routing.py</code> , <code>grounding.py</code>	One model, used consistently everywhere.
Embeddings	OpenAI API — text-embedding-3-small	<code>embeddings.py</code>	Used for chunk embeddings, query embeddings, and Stage 2a's summary-match.
Reranker (Priority 2)	bge-reranker-base , via <code>FlagEmbedding / sentence-transformers</code>	<code>reranker.py</code>	CPU-only, no GPU needed.
OCR / document parsing	Marker (open-source PDF→structured-markdown)	<code>ocr.py</code>	Used for every PDF, not just scans — plain text extraction loses table/heading/layout structure even on born-digital PDFs. Fallback if VRAM is tight: GOT-OCR2.0 .

8. Hardware Notes

Owner	Hardware needed	Why
P1 (ingestion)	NVIDIA GPU, 4-6GB VRAM minimum (e.g. RTX 4050)	Runs Marker/GOT-OCR2.0 locally. Falls back to CPU (much slower) or Google Colab's free GPU tier if unavailable.
Everyone else	Any standard machine, no GPU	DB access, embeddings, reranking (CPU), LLM calls, auth, frontend, eval all run on CPU or go through a network API call.

9. Refusal & Conflict — fixed rules and message templates

Confident answer: LLM answers using only the top chunks, with inline citation, e.g. `[Leave Policy 2025, Section 3.2.2]`.

Refusal (low confidence): - Priority 1 rule (before reranking exists): refuse if the top-1 dense retrieval score is below `0.72`, or zero chunks survive the metadata filter. - Priority 2 rule (once reranking exists): refuse if the top-1 reranked score is below threshold instead. - Either way, also ask the LLM to self-rate `high / medium / low` confidence that the passages support the answer; `low` also triggers refusal. - Template: *"I couldn't find a passage in the current policy documents that directly answers this. You may want to check with [department] or rephrase your question."*

Conflict detected (Priority 2): two chunks both tagged `version_status = "current"`, from different documents, give contradictory info on the same question. Never pick one silently. Template:

```
There appear to be two conflicting versions on file:
• [Document A] (effective [date]) states [value A]
• [Document B] (effective [date]) states [value B]
Please confirm which applies to your program, or flag this to the registrar —
both documents are currently marked active.
```

10. Coding Conventions (uniformity — this is what keeps 8 independently-generated codebases mergeable)

- **Language/framework:** Python 3.11+, FastAPI, `async / await` for all I/O (DB calls, HTTP calls, LLM calls).
- **Typing:** type hints on every function signature. All structured data crossing a function or API boundary is a Pydantic model from `schemas.py` — never a raw `dict`.
- **Docstrings:** one-line summary, then `Args: / Returns:` in Google style, on every public function.
- **Naming:** `snake_case` for functions/variables/files, `PascalCase` for classes and Pydantic models, `UPPER_SNAKE_CASE` for constants.
- **Config:** read all settings through `config.py`'s `Settings` object. Never call `os.environ` directly outside that file.
- **Errors:** raise typed exceptions defined in your own module; never let a raw, unhandled exception cross into another module's code. API-facing errors always return the shared `{ error, detail }` envelope from Section 5.
- **Logging:** Python's `logging` module, `logger = logging.getLogger(__name__)` at the top of each file. No `print()` statements in application code.
- **Imports:** absolute imports rooted at `app.` (e.g. `from app.retrieval.dense_retrieval import retrieve_chunks`), never relative

imports, never wildcard imports.

- **Function signatures:** must match Section 6 exactly — same parameter names, order, and types as given there. If a signature isn't specified there, define it narrowly and note it so the developer can confirm.
- **Tests:** pytest, one file per module (`tests/test_<module>.py`), using shared fixtures from `tests/conftest.py` (owned by P2) rather than inventing new fixture setups.
- **Formatting:** output should already match `black` (Python) / `prettier` (frontend) conventions — 4-space indents, double quotes in Python, trailing commas where `black` would add them.
- **Priority 2 code must fail safe:** wrap it so a missing dependency or a runtime error falls back to the Priority 1 behavior for that stage (Section 2), rather than raising an unhandled exception that breaks the whole request.

11. Per-Person Responsibilities (what each tag owns, not when)

Tag	Owns (files)	Priority 1 responsibility	Priority 2 responsibility
P1	<code>ingestion/*</code>	OCR-parse real documents into structured markdown (Marker), heading-aware chunking, metadata tagging, load chunks into the DB	Per-document summary, section-tree extraction, acronym/entity glossary — all feed P2's routing and P4's rewriting
P2	<code>database.py</code> , <code>config.py</code> , <code>deps.py</code> , <code>main.py</code> , <code>db/migrations/</code> , <code>retrieval/dense_retrieval.py</code> , <code>hybrid_retrieval.py</code> , <code>routing.py</code> , <code>retrieval/router.py</code>	Schema + migrations, dense retrieval with metadata filter as a hard constraint, <code>/retrieve</code> endpoint, integrates the whole backend	BM25 + RRF hybrid fusion, coarse routing (document candidate selection + section-tree reasoning)
P3	<code>retrieval/embeddings.py</code> , <code>indexer.py</code> , <code>reranker.py</code>	Embedding wrapper, batch indexing job	Cross-encoder reranking
P4	<code>generation/prompts.py</code> , <code>generator.py</code> , <code>query_rewriter.py</code> , <code>generation/router.py</code>	Grounded, cited answer generation; <code>/chat</code> endpoint; orchestrates the full pipeline call order (Section 6)	Query rewriting: acronym expansion, metadata predicate extraction, decomposition, clarify-then-fallback
P5	<code>generation/grounding.py</code> , <code>conflict_detector.py</code>	Refusal decision logic (confidence thresholds)	Version-conflict detection and dual-surfacing
P6	<code>auth/*</code> , <code>frontend/src/auth/*</code>	Login, JWT, tenant scoping, <code>get_current_user()</code>	Feedback capture endpoint
P7	<code>frontend/src/*</code> (excluding auth)	Chat UI: ask → streamed cited answer / clarifying question / refusal / conflict display	Feedback buttons, UI polish
P8	<code>eval/*</code> , <code>eval/gold_qa.json</code>	Gold Q&A set (30-50 questions), eval harness scoring retrieval hit-rate@k, answer faithfulness, hallucination rate	Failure-by-stage attribution report (routing / retrieval / generation)

12. Definition of Done — check before handing code back

- ☐ Only files owned by the requesting tag were created or modified (Section 3).
- ☐ Every function that other modules call matches its Section 6 signature exactly.
- ☐ Every piece of structured data uses a Pydantic model, not a raw dict, at any boundary.
- ☐ Priority 1 work is complete and correct before any Priority 2 work was attempted for that tag.
- ☐ Priority 2 code fails safe to its Priority 1 fallback if a dependency is missing or it raises.
- ☐ `tenant_id` is threaded through every DB query touched.
- ☐ No hardcoded config/secrets — everything goes through `config.py`'s `Settings`.
- ☐ A test file exists or was updated for the new logic, following the conventions in Section 10.